学習で当てる

ニューラル・バイグラム ― 「数える」と「学習する」が同じ答えに行き着く

rkskmt

前回の振り返り

- 数えて作った75×75の確率表で、架空の地名が作れた
- でも前コースの分類器は、表を**数えて作らなかった**

→ 損失を決めて、その損失が減る向きへ数値を少しずつ動かした ― **学習**

クイズ

前回の表と同じものを、数えずに**勾配降下法だけ**で作れるか？

- 作れる。しかも「数えられない問題」への扉がそこで開く ― 今日はその実験

問題を分類器の形に直す

- 「次の1文字を当てる」を、見慣れた形に並べ直す
- 分類器＝入力から**各選択肢の確率**を出す機械（**30秒リキャップ**）

	MNIST	今日
入力 x	画像（784次元）	いまの文字（1つ）
出力	10クラスの確率	**75クラスの確率**
訓練データ	(画像, 正解の数字)	(いまの文字, 次の文字)
損失	交差エントロピー	**交差エントロピー**

- 訓練データは前回のバイグラムの組そのもの：**11,428組**の (いまの文字, 次の文字)

道具箱の再点検 — PyTorch

- 今日から **PyTorch** を使う。基本の箱は **tensor** — NumPy 配列の親戚で、作り方も演算もほぼ同じ感覚

Code

```
import torch

a = torch.tensor([[1.0, 2.0], [3.0, 4.0]])
b = torch.tensor([10.0, 100.0])
print(a.shape)          # 形の確認も同じ
print(a @ b)            # 行列×ベクトルも @ 一発
```

Result

```
torch.Size([2, 2])
tensor([210., 430.])
```

- ここまでは NumPy と**見た目が違うだけ**。ではなぜわざわざ乗り換えるのか — tensor には決定的な特技が1つある

微分を自動で出す — autograd

- tensor は**自分がどう計算されたかを覚えている**。だから「結果を入力で微分した傾き」を自動で出せる

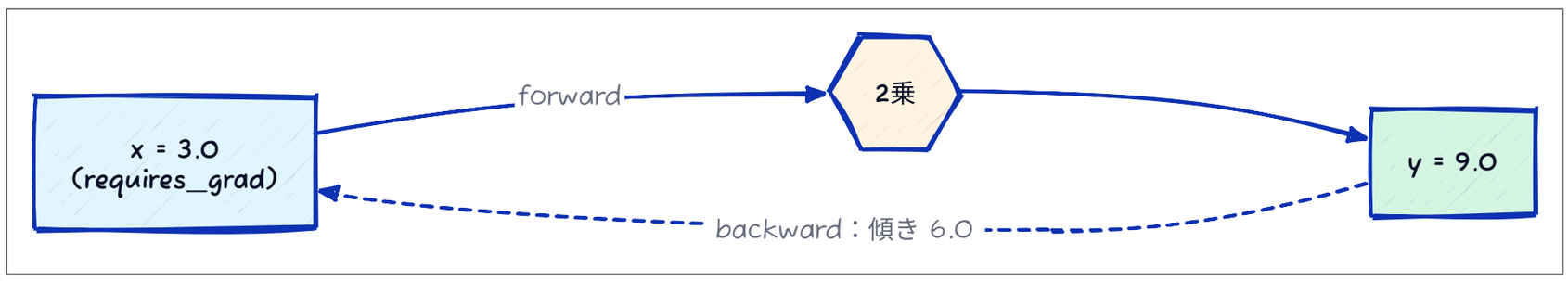
Code

```
import torch

x = torch.tensor(3.0, requires_grad=True)    # 「微分を追跡せよ」の印
y = x ** 2                                    # y = x²
y.backward()                                  # y を x で微分すると……
print(x.grad)                                 # dy/dx = 2x = 6 が出てくる
```

Result

```
tensor(6.)
```



- forward のたびに「どう計算したか」のメモ（計算グラフ）が残る。**`backward()`** は**メモを逆走**して傾きを届ける
- 微分の式 $2x$ を誰も教えていないのに、**傾き 6.0** が出た — この仕組みが **autograd**（自動微分）
- 何万個のパラメータから作った損失でも、同じ1行で**全員ぶんの傾き**が出る — 今日からの学習は全部これに乗る

文字を「ベクトル」にする — one-hot

- モデルに入れるために、文字番号を75次元のベクトルにする
- **one-hot ベクトル**：その文字の位置だけ 1、あとは全部 0

Code

```
import torch
import torch.nn.functional as F

x = torch.tensor([2, 0])                          # 文字番号 2 と 0
X = F.one_hot(x, num_classes=75).float()
print(X.shape)
print(X[0, :6], "...")                            # 2番の位置だけ 1
print(X[1, :6], "...")                            # 0番の位置だけ 1
```

Result

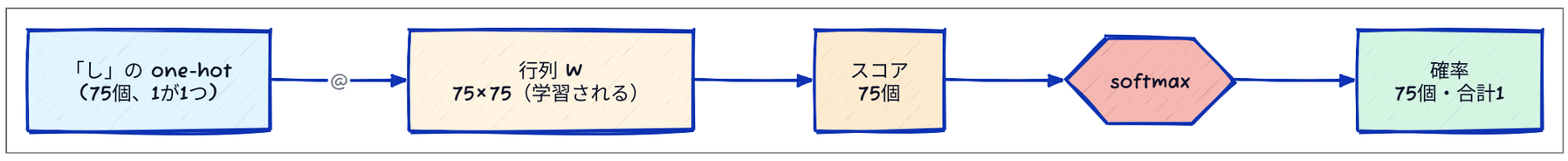
```
torch.Size([2, 75])
tensor([0., 0., 1., 0., 0., 0.]) ...
tensor([1., 0., 0., 0., 0., 0.]) ...
```

- 「どの文字か」という情報を、数値の演算に乗る形にしただけ

モデルは行列1枚

- パラメータは 75×75 の行列 W が1枚。one-hot を掛けて75個の**スコア**を出す（別名：**ロジット logit**）

$$\text{one-hot}(x) \mathbin{@} W \;=\; \text{75個のスコア} \xrightarrow{\text{softmax}} \text{確率}$$



- 掛けて足すだけの**線形**（**linear**）の変換1枚 — バイアスも活性化関数もなしの、最小の分類器
- スコアはまだ確率ではない：負の数もあるし、合計も1にならない — 確率に直す専門部品が**softmax**

スコアを確率に直す — softmax

- 欲しい変換：**全部正・合計1・スコアの大小はそのまま**
- 作り方は2手：e^s **を取る**（負でも正になる）→ **合計で割る**（合計が1になる）

$$\mathrm{softmax}(s_k) = \frac{e^{s_k}}{\sum_j e^{s_j}}$$

Code

```
import torch
import torch.nn.functional as F

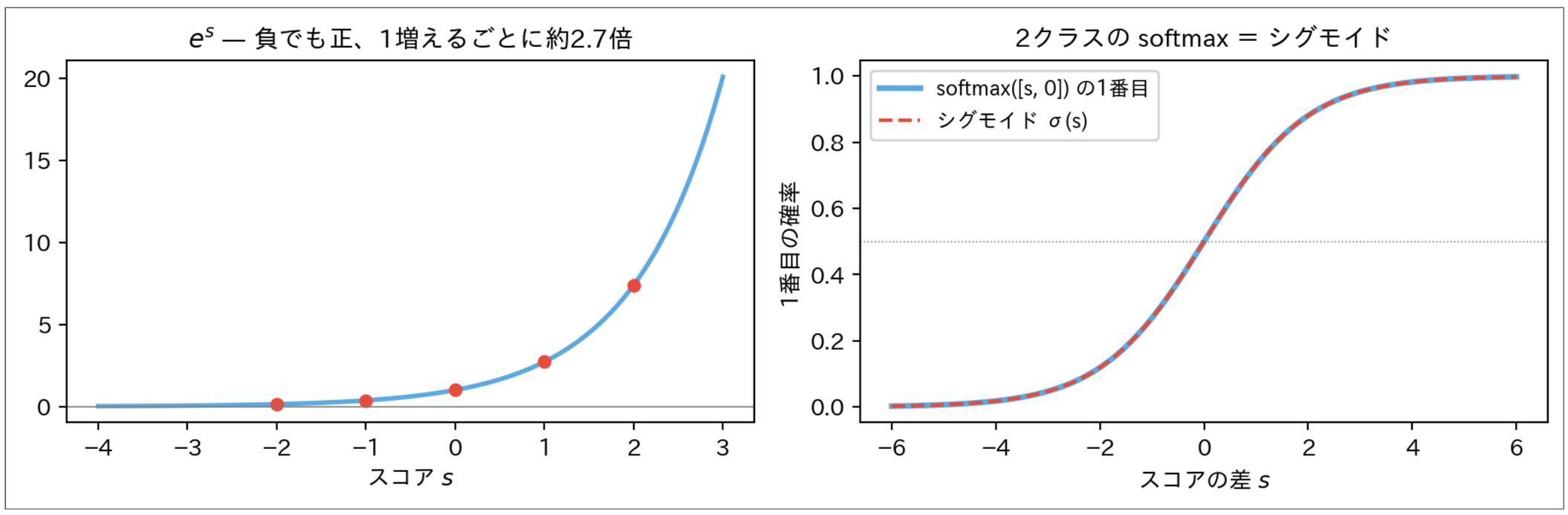
s = torch.tensor([2.0, 1.0, -1.0])      # スコア（負もある）
p = F.softmax(s, dim=0)
print(p)                                # 全部正・順位はそのまま
print(p.sum())                          # 合計はきっかり1
```

Result

```
tensor([0.7054, 0.2595, 0.0351])
tensor(1.0000)
```

- スコア (2, 1, −1) が確率 (0.71, 0.26, 0.04) になった。順位は保ちつつ、**差は強調される**
- 前コースの分類器の出力層に住んでいたのと同じ部品 — この講義では出ずっぱりになる

softmax の形 — e^s は差の増幅器



- 左：e^s はどんな負の数も正に直し、**スコアが1増えるだけで約2.7倍**にする増幅器
- だから確率の**比**はスコアの**差**だけで決まる：$p_i / p_j = e^{s_i - s_j}$ — 全員に同じ数を足しても出力は不変
- 右：クラスが2つなら softmax は**シグモイド**（前コースの2クラス分類の出力層に居たS字カーブ）と**完全に一致**

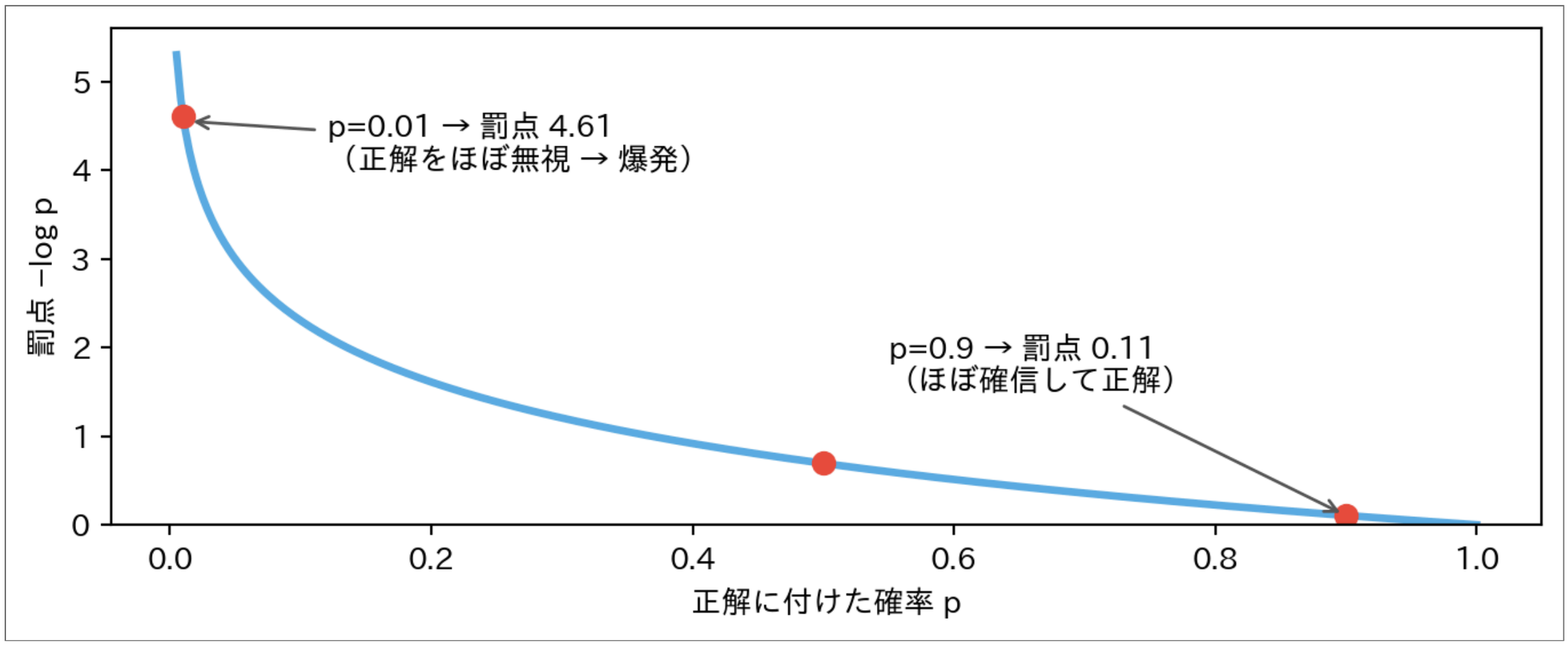
$$\mathrm{softmax}(s,\,0)_1 = \frac{e^s}{e^s + e^0} = \frac{1}{1 + e^{-s}} = \sigma(s)$$

- **softmax はシグモイドの多クラス版** — 新しい部品ではなく、知っている部品の一般形

答え合わせの罰点 — 交差エントロピー

- 「予想（確率分布）と正解1文字」の答え合わせ。ルールは1行：

$$\text{損失} = -\log\left(\text{正解に付けた確率}\right)$$



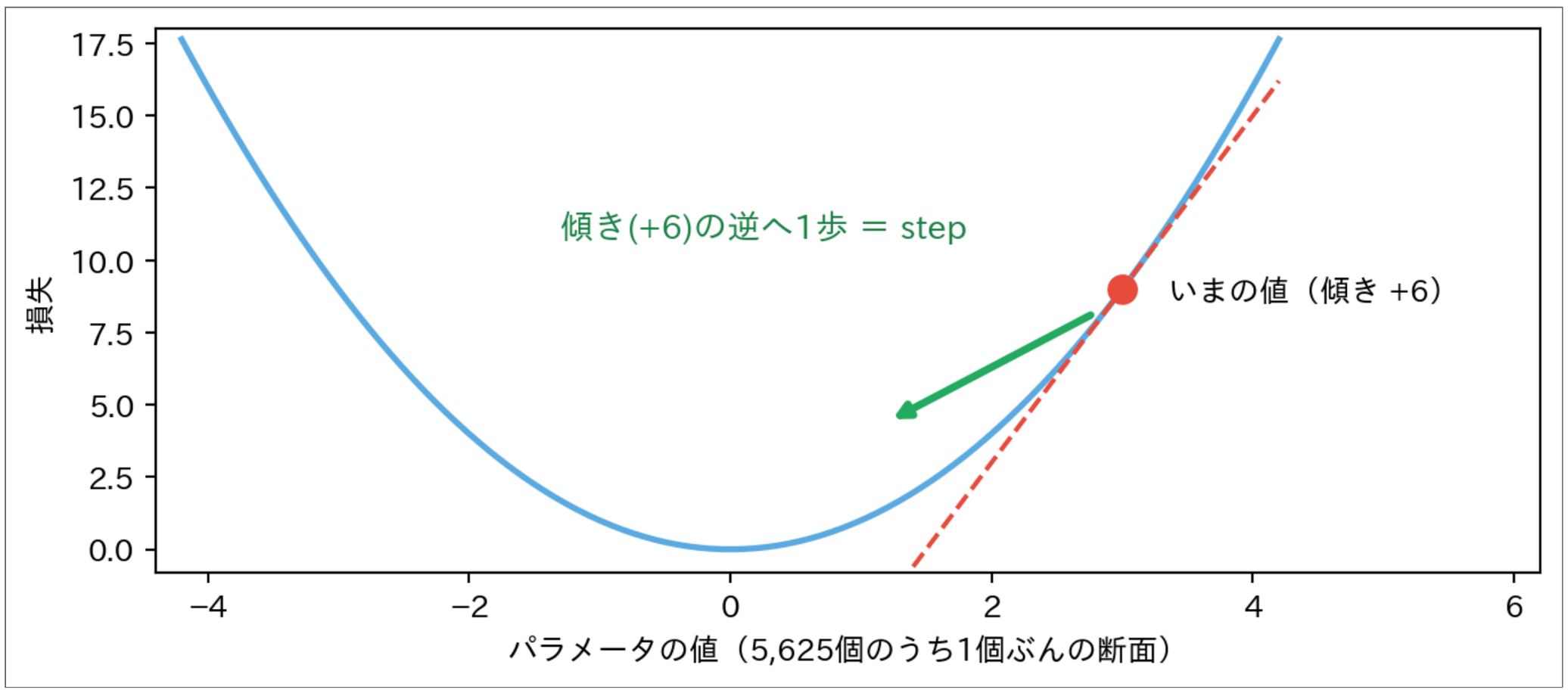
- 自信を持って当てればほぼ無罪（0.9 → 0.11）、五分五分で 0.69、正解を無視していたら重罪（0.01 → 4.61）— この採点法が**交差エントロピー**
- W を全部0で始めると全スコア0 → softmax は均等割りで各 $1/75$ → 罰点は $-\log(1/75) = 4.3175$

  → **学習を始める前に、最初の損失が予言できる**（数分後に答え合わせ）

> ⓘ **ノート**
>
> **`F.cross_entropy(スコア, 正解)`** は softmax とこの罰点を**まとめて**計算する関数。だからコードにはスコアを直接渡す。

学習＝損失の坂を下る

- 損失は W（5,625個の数）しだいで変わる。W をどっちに動かせば損失が減る？
- **`backward()`** が出す**勾配**（gradient）＝「どちらへ動かすと損失が増えるか」の矢印。**その逆へ少しずつ動かし続ける**のが**勾配降下法**（gradient descent）。1歩の幅が**学習率**（**`lr`**）



- さっきの autograd の「$x = 3$ で傾き 6」が、まさにこの図。損失でも同じことを**全パラメータの断面で一斉に**やる
- コードでは毎回この**3拍子**：

拍	コード	やっていること
①	**`opt.zero_grad()`**	前回の矢印を消す（消さないと足し込まれてしまう）
②	**`loss.backward()`**	さっきの **`x.grad`** を**全パラメータぶん一気に**（backprop）
③	**`opt.step()`**	矢印の逆へ1歩（勾配降下）

> ⓘ **ノート**
>
> オプティマイザ **AdamW** は、歩幅をパラメータごとに自動調整する改良版の勾配降下法。3拍子の型は変わらない。

学習ループを回す

Code

```
import pandas as pd
import torch
import torch.nn.functional as F

names = pd.read_csv("data/chimei.csv")["yomi"].tolist()
chars = ["・"] + sorted(set("".join(names)))
stoi = {c: i for i, c in enumerate(chars)}

xs, ys = [], []
for name in names:
    seq = "・" + name + "・"
    for a, b in zip(seq, seq[1:]):
        xs.append(stoi[a])
        ys.append(stoi[b])
X = F.one_hot(torch.tensor(xs), num_classes=75).float()
y = torch.tensor(ys)

W = torch.zeros((75, 75), requires_grad=True)
opt = torch.optim.AdamW([W], lr=0.5)
for it in range(1, 101):
    opt.zero_grad()                          # ① リセット
    loss = F.cross_entropy(X @ W, y)         # forward と損失
    loss.backward()                          # ② backprop（自動）
    opt.step()                               # ③ 勾配降下で1歩
    if it in (1, 10, 50, 100):
        print(f"更新 {it:3d}回 | Loss: {loss.item():.4f}")
```

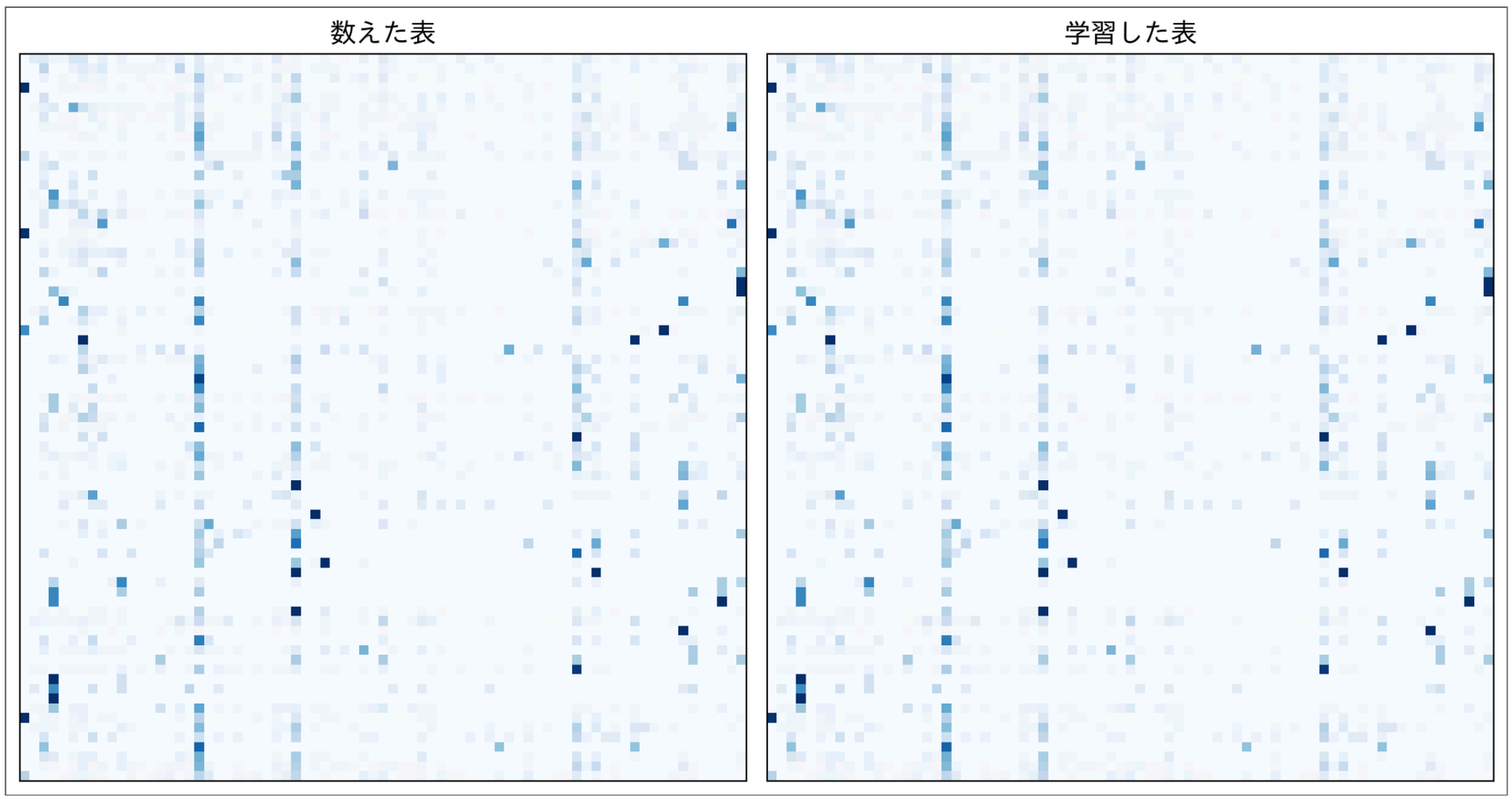
Result

```
更新   1回 | Loss: 4.3175
更新  10回 | Loss: 2.4495
更新  50回 | Loss: 2.3457
更新 100回 | Loss: 2.3460
```

- 1回目の損失は**予言どおり 4.3175**。そこから勾配降下で転がり落ちる

山場：2つの表を並べる

- 学習した W を softmax で確率にして、前回の「数えた表」と比べる



Code

```
print("確率の差の最大:", np.abs(P_learn - P_count).max().round(4))
print("P(う|ょ)  数え:", P_count[stoi['ょ'], stoi['う']].round(3),
      " 学習:", P_learn[stoi['ょ'], stoi['う']].round(3))
```

Result

```
確率の差の最大: 0.0067
P(う|ょ)  数え: 0.99  学習: 0.986
```

- **別々の道から、同じ表に着いた** — 勾配降下法は交差エントロピーが最小になる表を探し、その答えが「数えた表」だった

なぜわざわざ学習するのか

- 数える方が速くて正確なのに、遠回りした理由：

クイズ

「直前の**5文字**を見て次を当てる」を表で作ると、行はいくつ必要？

- $75^5 \approx$ **23億行**。データは9,681文字しかない — ほぼ全行が空っぽになる
- **数える作戦は文脈を伸ばした瞬間に破綻する**。勾配降下法はこの先も使える
- ここから先のモデル（MLP、そして Transformer）は全部この3拍子で学習する

学習した表でも生成できる

- 前回の生成コードの **P** を **P_learn** に差し替えるだけ

Code

```
import numpy as np

rng = np.random.default_rng(0)
for _ in range(5):
    i, s = 0, ""
    while True:
        i = rng.choice(75, p=P_learn[i] / P_learn[i].sum())   # 丸め誤差の補正
        if i == 0:
            break
        s += chars[i]
    print(s)
```

Result

```
とさいいままちょうむら
みいなかわち
さいさとべつしんらし
なごうし
きたやまちょう
```

- 出来ばえも前回と同レベル — 中身が同じ表なのだから当然

Tweak 学習の挙動を観察する



- **学習ループのコード** を書き換えて実行

1	**lr=0.5 → lr=0.01**	100回では全然下がりきらない（何回ならいい？）
2	**range(1, 101) → range(1, 501)**	損失はどこまで下がる？ 2.34 の壁を観察
3	**torch.zeros → torch.randn(75, 75, requires_grad=True)**	初期損失が 4.3175 からずれる（一様分布でなくなる）

ここで観察

- どう頑張っても損失は**約2.34で頭打ち**になる。これはコードのバグでも学習不足でもない — 次の練習問題で正体を暴く

まとめ

- 「次の1文字」を**75クラス分類**に直した：one-hot × 行列 W → softmax → 交差エントロピー
- 勾配降下法で学習した表は、**数えた表と一致**した（差は小数第3位以下）
- 数える作戦は文脈を伸ばすと破綻する。学習はこの先も通用する — だから遠回りに価値がある
- 学習ループは **`zero_grad → backward → step`** の**3拍子**：autograd が全パラメータの傾きを出し、その逆へ1歩 — この型はこの先ずっと同じ

現在の設計図　文字番号 → one-hot → ==学習する行列 W== → softmax → 次の文字

ⓘ 次回予告

one-hot × 行列の計算、実は**行を1本選んでいるだけ**。 この見方から「文字の意味をベクトルで持つ」というLLMの心臓部が生まれる。

練習問題

以下の問題を**Pythonコード**で解け。

問題0（ウォームアップ）：softmax と罰点の感覚

コードを書く前に頭で答え、そのあと1行ずつコードで検算せよ。

1. スコア $(5, 5, 5)$ を softmax に通すと出てくる確率は？
2. 正解に確率 1.0 を付けたときの交差エントロピーは？
3. スコア全員に $+100$ して $(102, 101, 99)$ にすると、softmax の出力は $(2, 1, -1)$ のときから変わるか？

問題1：損失2.34の壁の正体

「数えた表」そのものの損失を計算し、学習の到達点と一致することを確かめよ。

0. 本編のコードで **`P_count`** と **`xs`**, **`ys`** まで作る
1. 全11,428組について $-\log P_{\text{count}}[x, y]$ の平均を計算（**`np.log`** と配列添字を使う）
2. 学習後の損失（約2.346）と比べる

問題2：どの文字が一番「読みにくい」か

文字ごとに「その文字の次を当てるときの平均損失」を計算し、**次が一番予想しにくい文字**と**一番予想しやすい文字**を1つずつ挙げよ。

1. 各組 (x, y) の損失 $-\log P_{\text{count}}[x, y]$ を、x の文字ごとに平均する
2. 平均損失が最大の文字と最小の文字を表示（登場10回未満の文字は除く）
3. 最小の文字がなぜ予想しやすいか、表の行を見て説明せよ

解答例

▶ 解答例を見る

Result

```
tensor([0.3333, 0.3333, 0.3333])
tensor(-0.)
tensor([0.7054, 0.2595, 0.0351])
数えた表の損失: 2.3433
予想しにくい: ・ (3.465)
予想しやすい: ょ (0.070)
```

- 問題0：同点なら均等割り、全員に同じ数を足しても出力は不変 — softmax は**スコアの「差」だけ**を見ている
- 問題1：学習は「数えた表」を再発見しただけ — **2.34はバイグラムという作戦自体の限界**。これより下げるには、1文字より多くの文脈を見るしかない（次回以降の伏線）
- 問題2：最も当てやすいのは「ょ」（ほぼ「う」一択、平均損失0.07）。最難関は境界「・」（3.47）— **名前の1文字目が一番予想がつかない**、という直感どおりの結果